

Reporte del proyecto: The Uno

Versión 2: juego de UNO para 3 jugadores en red local (LAN)

Integrantes

Tonatiuh Charles Pérez Minssart

Héctor Rodrigo Benítez Gama

Alejandro Hernández Hernández

Interfaces Gráficas con Aplicaciones, 223602

Profesor: Alberto Salvador Núñez Varela

08/11/2026

1. Descripción general del proyecto

The Uno es un juego de UNO para 3 jugadores, uno por cada integrante del equipo, desarrollado en C# con Avalonia y el patrón MVVM. En esta segunda versión cada integrante juega desde su propia computadora, conectados en la misma red local (LAN): una computadora crea la partida y funciona como servidor, y las otras dos se unen con su IP. Cada quien solo ve sus propias cartas, y las partidas, el historial y todos los movimientos se guardan en SQLite por medio de una API REST en Python con FastAPI.

Seguimos las reglas oficiales de Mattel: cada jugador recibe 7 cartas de una baraja de 108; se puede tirar una carta del mismo color, número o símbolo, o un comodín; el Salto hace perder el turno al siguiente jugador; la Reversa invierte el sentido del juego (en 3 jugadores no funciona como Salto); el +2 y el +4 obligan al siguiente jugador a robar y perder su turno, sin acumulación; el +4 solo se puede tirar si el jugador no tiene ninguna carta del color activo; y quien no canta UNO al quedarse con una carta recibe 2 cartas de penalización.

1.1 Arquitectura

Para la versión en red seguimos el plan de migración que acordamos el 30 de octubre: la solución pasó a tener cuatro proyectos y un proyecto de pruebas, y cada integrante quedó a cargo de un nodo de la red. Los proyectos se comunican por medio de contratos definidos en el Core y del contrato del hub de SignalR:

- **TheUno.Core:** el motor de juego en C# puro. Contiene los modelos (carta, jugador y baraja), las reglas, la máquina de turnos y los eventos del juego. En la versión en red corre dentro del servidor, así que hay una sola partida y unas mismas reglas para las tres computadoras.
- **TheUno.Server:** el servidor central, con ASP.NET Core y SignalR. Mantiene el motor en memoria, recibe las jugadas de los clientes por medio del GameHub y le manda a cada uno su propia vista de la mesa. Escucha en toda la red en el puerto 5050.
- **TheUno.UI:** el cliente en Avalonia con estética de 8 bits. Se conecta al servidor con SignalR, manda las acciones del jugador (tirar, robar, pasar, cantar UNO) y dibuja el estado que recibe: es un cliente pasivo que ya no decide las reglas.
- **TheUno.Infrastructure:** la persistencia. Incluye la API en Python (FastAPI + SQLite), que ahora escucha en toda la red en el puerto 8000, los repositorios que la consumen desde C#, el servicio de auditoría y la versión con Entity Framework Core con la que se diseñó la base de datos.
- **TheUno.Tests:** pruebas unitarias del motor y del servidor, y una prueba de integración que conecta tres clientes SignalR al GameHub.

Cuando el anfitrión inicia la partida, el servidor la registra en la API con los tres jugadores; cada movimiento se guarda en la bitácora y, al terminar, se guardan el ganador, el número de turnos y

la duración. Cada cliente recibe del servidor una mesa distinta: la suya con sus cartas y la de los rivales solo con cuántas cartas tienen.

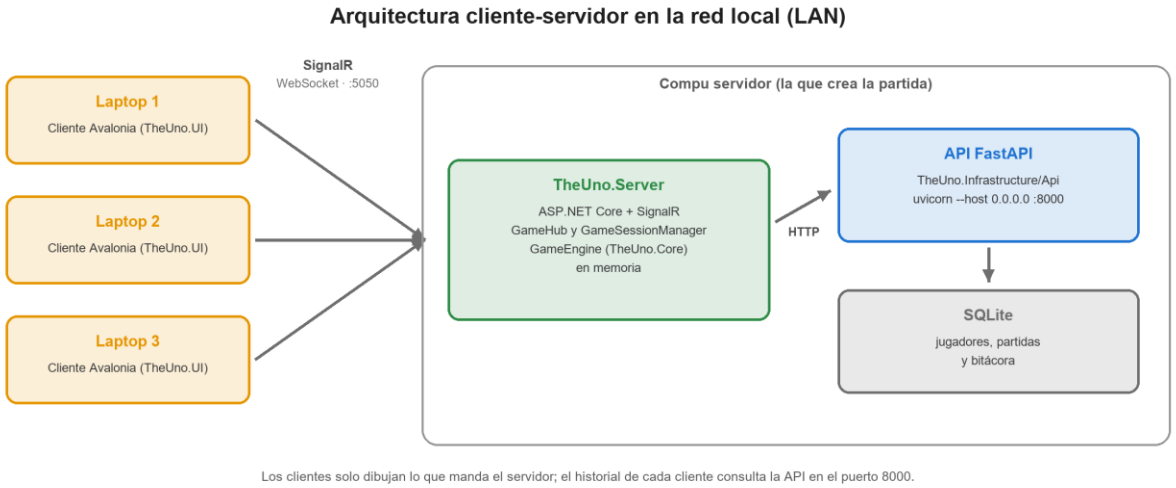


Figura 1. *Arquitectura cliente-servidor en la red local: tres clientes Avalonia conectados al servidor SignalR, que guarda las partidas en la API.*

1.2 Tecnologías utilizadas

Tecnología	Uso en el proyecto
C# y .NET 10	Lenguaje y plataforma de toda la solución.
Avalonia 12 y CommunityToolkit.Mvvm	Interfaz gráfica multiplataforma con el patrón MVVM (propiedades observables y comandos).
ASP.NET Core y SignalR	Servidor central del juego: conexión en tiempo real por WebSockets entre las tres computadoras de la red local.
xUnit	Pruebas unitarias del motor y del servidor, y prueba de integración con tres clientes SignalR.
NAudio / afplay	Música de fondo y efectos de sonido en Windows (NAudio) y en macOS (afplay).
Python, FastAPI y SQLite	API REST que guarda jugadores, partidas y movimientos; escucha en toda la red local para el servidor y el historial.
Entity Framework Core	Diseño de la base de datos: entidades, migraciones y semilla de jugadores.
Git y GitHub	Control de versiones, una rama por área (Core, Infrastructure, Interfaz y Server) y pull requests hacia main.

1.3 Capturas del juego



Figura 2. *Menú principal con los botones para iniciar el juego y ver el historial.*

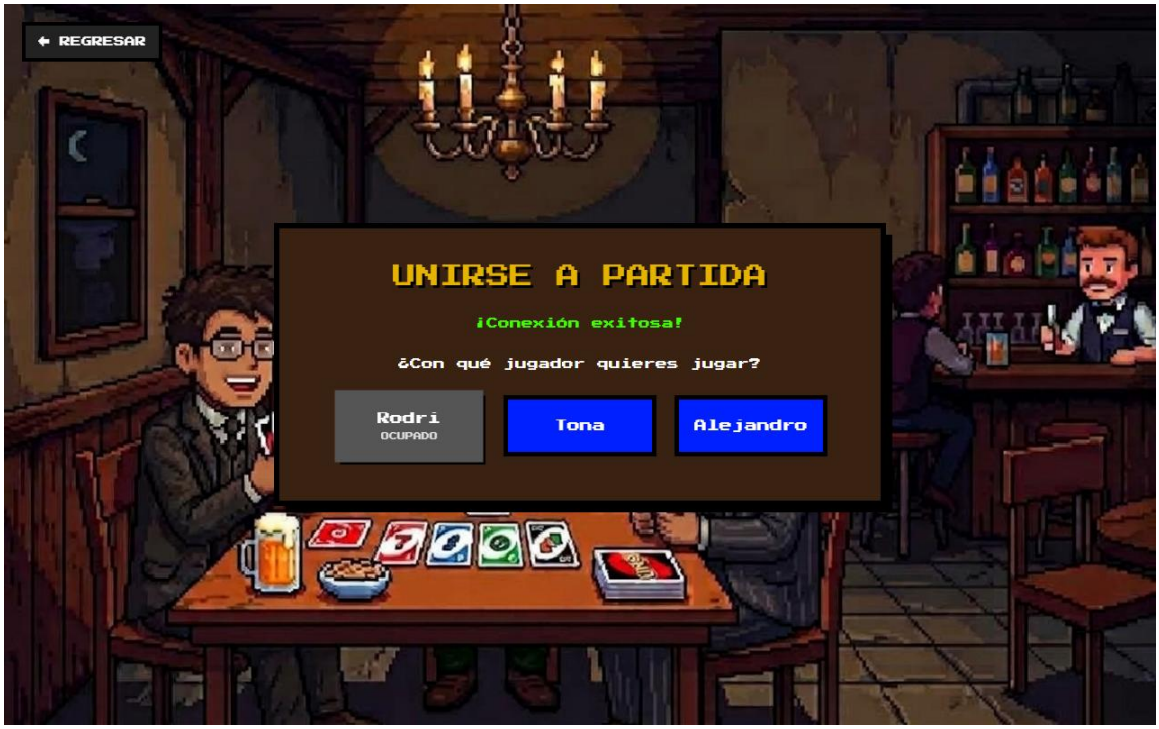


Figura 3. *Unirse a partida: después de conectarse a la IP del servidor se elige jugador; el que ya eligió alguien más aparece como OCUPADO.*



Figura 4. *Sala de conexión del anfitrión con los tres jugadores. Solo quien creó la partida la inicia, y cualquiera puede cambiar de jugador antes de empezar.*



Figura 5. *Partida en red vista desde la computadora de Tona: el jugador en turno aparece en mayúsculas y en amarillo, sus cartas se asoman mientras elige y todos ven el aviso cuando alguien canta UNO.*



Figura 6. *Aviso de jugada: cuando alguien tira un +2, +4, saltar o reversa, las tres computadoras ven quién pierde el turno.*



Figura 7. *Pantalla de victoria con el ganador y la duración de la partida.*



Figura 8. *Historial: tabla de clasificación, lista de partidas terminadas y bitácora con los movimientos de la partida elegida.*

2. División de tareas entre los integrantes del equipo

El trabajo se dividió en dos fases. En la primera (22 de septiembre al 10 de octubre) cada integrante desarrolló su proyecto de la solución para el juego local. En la segunda (30 de octubre al 8 de noviembre) seguimos el plan de migración multijugador, en el que cada integrante asumió el control de un nodo de la red. En las dos fases trabajamos en ramas propias e integramos los cambios a main por medio de pull requests revisados.

Integrante	Rol	Responsabilidades principales
Héctor Rodrigo Benítez Gama (Rodri)	Motor de juego y servidor central (TheUno.Core, TheUno.Server y TheUno.Tests)	Fase 1: estructura de la solución, contratos, modelos, reglas oficiales, motor y eventos. Fase 2: servidor ASP.NET Core con SignalR, GameHub, sesión de juego y pruebas.
Tonatiuh Charles Pérez Minssart (Tona)	Interfaz y cliente en red (TheUno.UI)	Fase 1: tablero, recursos gráficos, menú, música, diálogos e historial. Fase 2: conexión del cliente con SignalR, sala de conexión, manos privadas y avisos de juego.
Alejandro Hernández Hernández	Persistencia en red y auditoría (TheUno.Infrastructure)	Fase 1: diseño de la base de datos, migraciones, semilla, repositorios, auditoría y API en Python. Fase 2: auditoría en el servidor, API expuesta en la red, clientes HTTP y contratos JSON.

2.1 Fase 1 · Rodri: motor de juego (TheUno.Core y TheUno.Tests)

- **Estructura y contratos:** creó la solución con proyectos desacoplados y definió las interfaces y DTOs compartidos (IGameEngine, IGameLogRepository, IMatchRepository y GameLogEntryDto).
- **Modelos y reglas:** Card, Player y Deck (108 cartas, barajado y reciclaje), y las reglas oficiales en GameRules: jugada legal, restricción del +4 y sin acumulación de sanciones.
- **Motor y eventos:** turnos para 3 jugadores, cambio de sentido, cartas especiales, regla de UNO y eventos (OnCardPlayed, OnTurnChanged, OnGameOver, etc.) para avisar a la interfaz y a la auditoría.
- **Pruebas unitarias:** suite con xUnit y un repositorio en memoria para validar la baraja, los turnos y las reglas.

2.2 Fase 1 · Tona: interfaz y experiencia de usuario (TheUno.UI)

- **Tablero y recursos gráficos:** mesa con los tres jugadores, mazo, descarte, color activo y sentido del juego, con cartas en pixel art, fondos y la fuente Press Start 2P.
- **Menú y música:** menú principal y música para el menú y la partida en Windows y macOS.

- **Conexión con el motor y diálogos:** comandos para tirar, robar y pasar; selector de color, botón ¡UNO!, penalización y pantalla de victoria.
- **Historial:** tabla de clasificación, partidas terminadas y bitácora de movimientos, leídas de la API.

2.3 Fase 1 · Alejandro: base de datos y persistencia (TheUno.Infrastructure)

- **Modelo de datos:** entidades Player, Match, MatchPlayer y GameLog con Entity Framework Core, migraciones y la semilla con los tres integrantes.
- **Repositorios y auditoría:** MatchRepository y GameLogRepository, y GameAuditService, que convierte los eventos del motor en registros de la bitácora.
- **API en Python:** API REST con FastAPI y SQLite para crear y cerrar partidas, guardar movimientos y consultar estadísticas, y los repositorios en C# que la consumen.
- **Endpoints de la API:**

Endpoint	Descripción
GET /	Solo para comprobar el estado del servidor.
POST /matches	Registra el inicio de una nueva partida y vincula a los jugadores participantes en la base de datos.
POST /matches/{mathc_id/complete	Marca una partida existente como completada
POST /logs	Almacena una entrada de auditoría para registrar eventos durante la partida.
GET /player/stats	Devuelve una tabla de clasificación con las estadísticas acumuladas de todos los jugadores registrados.
GET /matches	Retorna el historial de todas las partidas finalizadas
GET /matches/{math_id}/logs	Devuelve todos los movimientos y eventos registrados para una partida específica.

- **Desacoplamiento de EF Core:** EF Core se usó para diseñar el modelo y la semilla; la persistencia activa quedó en la API de Python, que C# consume con HttpClient.

2.4 Fase 2: migración a red local (plan del 30 de octubre)

Cada integrante tomó un nodo de la red, como se acordó en el plan de migración. Las tres partes avanzaron en paralelo y se conectaron por medio del contrato del hub y de los contratos JSON de la API.

Rodri: servidor central y lógica de red (TheUno.Server)

- **Servidor central:** creó TheUno.Server con ASP.NET Core, escuchando en toda la red en el puerto 5050, con GameSessionManager para mantener el motor en memoria y controlar la sala.

- **GameHub con SignalR:** métodos para crear partida, elegir jugador, tirar, robar, pasar y cantar UNO, y mensajes de difusión en tiempo real a todos los clientes.
- **Estado por jugador:** cada cliente recibe su propia mesa con su mano; de los rivales solo cuántas cartas tienen. La ventana de 2 segundos para cantar UNO la lleva el servidor.
- **Pruebas:** pruebas unitarias del servidor y una de integración con tres clientes SignalR contra el GameHub.

Tona: cliente de interfaz gráfica (TheUno.UI)

- **Conexión con SignalR:** servicios que mantienen la conexión con la IP del servidor y el contrato del hub en GameContracts.
- **Tablero pasivo:** BoardViewModel ya no usa el motor local: dibuja lo que manda el servidor, con los rivales boca abajo, y los clics invocan los métodos del GameHub.
- **Pantallas de red y experiencia:** crear o unirse a partida, sala de conexión, cambiar de jugador, volver a jugar, avisos de jugadas y animaciones del tablero.

Alejandro: persistencia en red y auditoría (TheUno.Infrastructure)

- **Auditoría en el servidor:** trasladó GameAuditService al servidor, donde ahora vive el motor.
- **API en toda la red:** arranque con uvicorn api:app --host 0.0.0.0 --port 8000 para que la consulten el servidor y el historial.
- **Clientes HTTP:** movió AddHttpClient de ApiMatchRepository y ApiGameLogRepository de Avalonia al Program.cs del servidor.
- **Contratos JSON y documentación:** compatibilidad de los payloads sin cambiar la base ni los endpoints, y la IP de la API en el README.

3. Diagrama entidad-relación de la base de datos

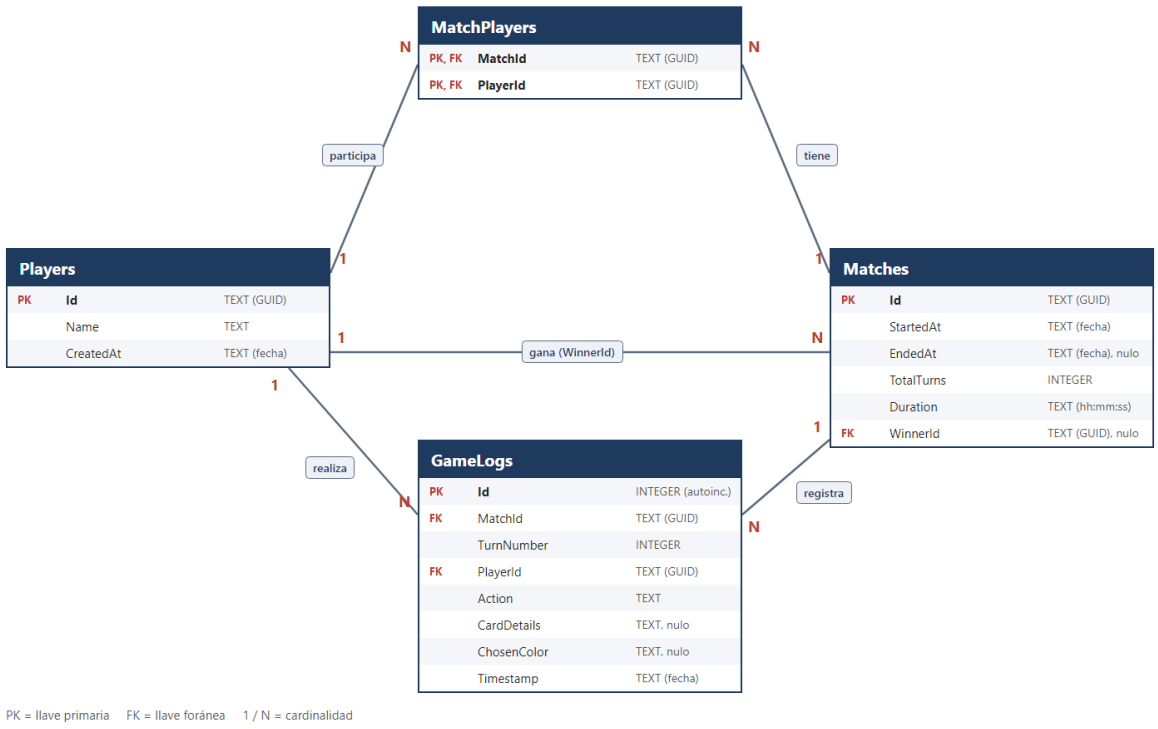


Figura 9. *Diagrama entidad-relación de la base de datos SQLite que usa la API.*

Tabla	Descripción
Players	Jugadores registrados. Contiene a los tres integrantes del equipo desde la semilla, por lo que no hace falta un formulario de registro.
Matches	Una fila por partida: inicio, fin, número de turnos, duración y ganador (WinnerId, llave foránea a Players). Una partida sin fecha de fin es una partida que no se terminó.
MatchPlayers	Relación muchos a muchos entre partidas y jugadores: qué jugadores participaron en cada partida. Su llave primaria es compuesta (MatchId, PlayerId).
GameLogs	Log del juego: cada movimiento de una partida con su número de turno, el jugador, la acción (tirar, robar, pasar, cantar UNO, penalización o fin de partida), la carta y el color elegido.

Relaciones: un jugador puede ganar muchas partidas (1:N); partidas y jugadores se relacionan de muchos a muchos por medio de MatchPlayers; y cada partida y cada jugador tienen muchos movimientos en GameLogs (1:N). Con estas tablas se obtiene el historial de partidas ganadas y perdidas por jugador y la traza completa de cada partida.

4. Listado de commits

Commits de la rama main del 22 de septiembre al 8 de noviembre de 2026: 201 commits en total, incluidos los merges de los pull requests. La figura siguiente muestra la línea de tiempo de las ramas: cada punto es un commit, cada círculo blanco un pull request integrado a main y las líneas grises los momentos en que cada integrante trajo main a su rama. Algunos integrantes aparecen con dos nombres de autor porque hicieron commits desde distintas computadoras o desde GitHub.

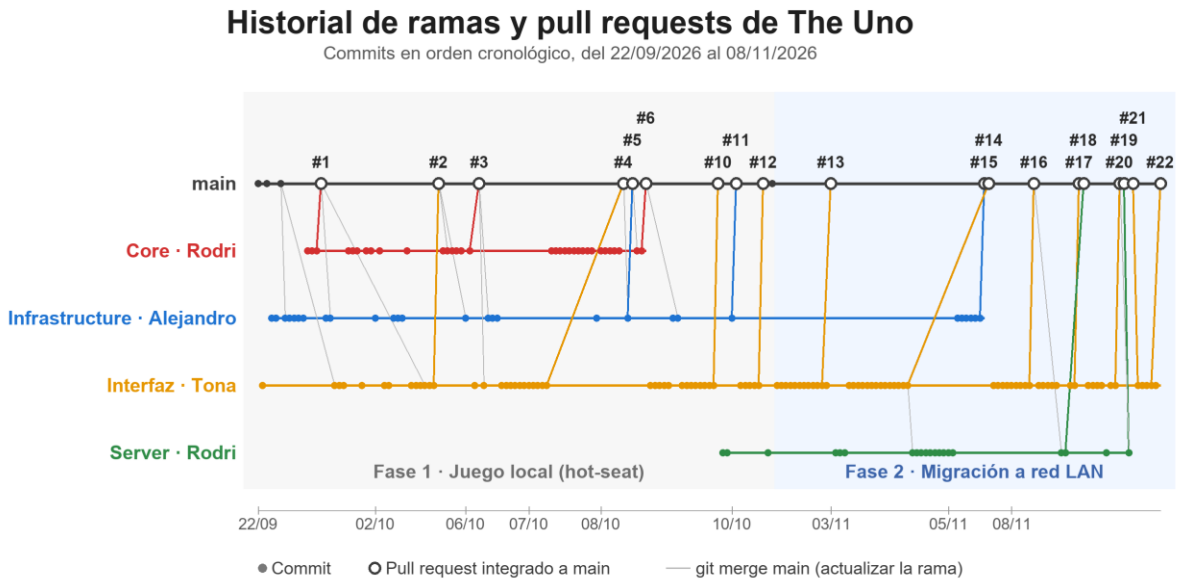


Figura 10. *Línea de tiempo de las ramas y los pull requests del proyecto, en las dos fases.*

Fecha	Autor	Commit	Mensaje
22/09/2026 11:33	Rodrigo Benitez	c4a0b	Normalizar saltos de linea con .gitattributes
22/09/2026 11:45	Soy Tonatiuh	0f66c	Prueba Inicial Push Branch
25/09/2026 00:18	Rodrigo Benitez	a2cb8	bases de la arquitectura del sistema con una sln con tres proj, Core, Infrastructure, UI
25/09/2026 18:43	Alejandro	aabbc	Se añadió a Infrastructure: clases {Player, Match, GameLog} como entidades, con sus respectivos atributos y propiedades
25/09/2026 19:25	Alejandro	23efd	Se añadió a infrastructure, {UnoDbContext] y se inicio La migración de EF Core, se generó la semilla principal con los tres jugadores ya predeterminados
27/09/2026 22:23	Rodrigo Benitez	3b965	feat: Implementacion de los contratos e interfaces para la infrastructura
28/09/2026 17:27	Alejandro	a471a	Merge branch 'main' of https://github.com/rodrigo-bg/TheUno into Infrastructure
28/09/2026 17:27	Alejandro	be338	feat(infrastructure): Guarde el trabajo para el pull

Fecha	Autor	Commit	Mensaje
28/09/2026 18:15	Alejandro	6f74e	feat(infrastructure): Implementa MatchRepository y GameLogRepository usando Core Contracts y Guid, se actualizo las entidades Player, Match Y GameLog porque tenian un formato diferente al implementado en Core
28/09/2026 19:08	Alejandro	dd9ec	feat: Agregue la carpeta de Services para implementar GameAuditService, esperando TheUno.Core.Engine
30/09/2026 20:44	Alejandro	14af3	Infrastructure, se añadió GameAuditService, aqui se gestiona el los eventos que suceden en una partida, lanzar carta, reversa, etcetera y se guardan en la base de datos
01/10/2026 11:37	Rodrigo Benitez	e5848	Defincion de los modelos de Cartas y Jugador
01/10/2026 11:42	Rodrigo Benitez	bcef0	Se definen los posibles eventos de el juego
01/10/2026 11:55	Rodrigo Benitez	e1acb	feat:(core) Agrego las interfaces y contratos necesarios para la UI
01/10/2026 12:00	Héctor Rodrigo Benítez Gama	4472b	Merge pull request #1 from rodrigo-bg/Core
01/10/2026 12:04	Alejandro	02190	feat(infrastructure): Implemente las plnatillas para la api y app que conecta con la base de datos
01/10/2026 12:06	Alejandro	89825	Merge branch 'main' into Infrastructure
01/10/2026 20:21	Soy Tonatiuh	f06f4	FIX(UI): Resolver conflictos de merge con main y ajustar el proyecto de interfaz
01/10/2026 20:32	Soy Tonatiuh	407dc	FEAT(UI): Agregar recursos graficos de las cartas al proyecto de interfaz
01/10/2026 20:47	Soy Tonatiuh	d86fa	FEAT(UI): Construir la estructura del tablero principal con los tres jugadores y la zona central
01/10/2026 21:17	Rodrigo Benitez	1c037	refactor: Modifiqué la clase Card para darle mas legibilidad
01/10/2026 21:22	Rodrigo Benitez	97fa2	feat(core): metodo para asignar valor a cartas y refactorizacion de GetDisplayText
01/10/2026 21:25	Rodrigo Benitez	b6676	refactor(Player): implementacion de setters y getters explicitos
01/10/2026 21:25	Soy Tonatiuh	33b32	FEAT(UI): Mostrar las manos y el estado de la mesa con datos de ejemplo
01/10/2026 21:44	Rodrigo Benitez	e2cc6	feat(core): agrega segundo constructor y calculo de puntos a Player.cs
01/10/2026 21:49	Rodrigo Benitez	6d7ed	feat(Models): agrega la clase Deck.cs e implementa el constructor
02/10/2026 11:06	Alejandro	83bd6	Merge branch 'Infrastructure' of https://github.com/rodrigo-bg/TheUno
02/10/2026 11:36	Rodrigo Benitez	26ed3	feat(Deck): agrega funciones getter de Deck.cs
02/10/2026 15:47	Soy Tonatiuh	2cff9	FEAT(UI): Cambiar el diseño de las cartas al estilo 8 bits en formato PNG
02/10/2026 16:16	Soy Tonatiuh	13069	FEAT(UI): Aplicar la tematica 8 bits al tablero con fondo, tipografia y paleta de colores
03/10/2026 15:26	Alejandro	3ed85	feat(infrastructure): Se creo la api para guardar y consultar datos, adema se refacrotizo GameAuditService por conflictos con los nombres de contracts y de los eventos
03/10/2026 16:00	Alejandro	f5b7a	fix(Infrastructure): Resolvi un problema con la migracion por el merge
03/10/2026 16:03	Alejandro	c6543	fix(Infrastructure): Resolvi un problema con la migracion por el merge, se arreglo problemas en GaeAuditService por nombres desiguales y ademas se implemento la api
04/10/2026 22:20	Rodrigo Benitez	132da	feat(deck): metodos de la clase Deck.csv (shufle, resset, draw, discard)

Fecha	Autor	Commit	Mensaje
05/10/2026 19:41	Soy Tonatiuh	5a945	FEAT(UI): Agregar menu principal con navegacion al tablero y tipografia pixel en toda la interfaz
05/10/2026 19:59	Soy Tonatiuh	02935	FIX(UI): Orientar las cartas del jugador 3 hacia su perspectiva
05/10/2026 19:59	Soy Tonatiuh	a4486	FEAT(UI): Agregar musica de fondo en loop durante el menu y la partida
05/10/2026 20:11	Soy Tonatiuh	736c5	FIX(UI): Integrar los cambios de main con los modelos y contratos del Core
05/10/2026 20:15	Soy Tonatiuh	93f13	FEAT(UI): Conectar las cartas de la interfaz con el modelo Card del Core
05/10/2026 20:31	Soy Tonatiuh	e6ef6	FIX(UI): Reemplazar LibVLC para que la musica funcione en Windows y Mac
05/10/2026 22:52	Héctor Rodrigo Benítez Gama	5ed41	Merge pull request #2 from tonatiuh/Interfaz
05/10/2026 22:56	Rodrigo Benitez	97514	cambios antes del pull request
05/10/2026 23:00	Rodrigo Benitez	41cca	Merge branch 'main' into Core # Please enter a commit message to explain why this merge is necessary, # especially if it merges an updated upstream into a topic branch. # # Lines starting with '#' will be ignored, and an empty message aborts # the commit.
05/10/2026 23:08	Rodrigo Benitez	2aafc	feat(GameRules): Agreaga el metodo en la clase GameRules.cs el cual valida si la carta que se quiere tirar es valida
05/10/2026 23:13	Rodrigo Benitez	d50a2	feat(GameRules): Agrega metodo que determina si puede jugar un +4 o tiene al menos una jugada legal
05/10/2026 23:22	Rodrigo Benitez	219b3	feat(GameRules): Agrega metodo para evitar que algun jugador esté stackeando +2 o +4
06/10/2026 11:15	Alejandro	8db45	Merge branch 'main' of https://github.com/rodrigo-bg/TheUno
06/10/2026 11:38	Rodrigo Benitez	aed7d	fix(Card): Se agrega un metodo que se esperaba en CardViewModel.cs
06/10/2026 11:39	Soy Tonatiuh	e8fef	FEAT(UI): Usar una cancion distinta para el menu y para la partida
06/10/2026 11:44	Héctor Rodrigo Benítez Gama	37750	Merge pull request #3 from rodrigo-bg/Core
06/10/2026 11:47	Soy Tonatiuh	43ce7	FIX(UI): Integrar los cambios del Core con las reglas y la baraja
06/10/2026 11:52	Alejandro	2e038	Merge branch 'main' of https://github.com/rodrigo-bg/TheUno
06/10/2026 11:52	Alejandro	8a4d7	feat(infrastructure): Agregue la clase de MatchPlayer para guardar los jugadores de cada partida
06/10/2026 11:53	Alejandro	fa290	Merge branch 'main' into Infrastructure # Please enter a commit message to explain why this merge is necessary, # especially if it merges an updated upstream into a topic branch. # # Lines starting with '#' will be ignored, and an empty message aborts # the commit.
06/10/2026 19:27	Soy Tonatiuh	ffb5e	FEAT(UI): Resaltar al jugador en turno y atenuar las manos de los demas
06/10/2026 19:46	Soy Tonatiuh	b6e68	FEAT(UI): Resaltar la carta al pasar el mouse sobre la mano del jugador en turno
06/10/2026 20:05	Soy Tonatiuh	20501	FIX(UI): Quitar el marco del jugador en turno y dejar solo su nombre resaltado
06/10/2026 20:05	Soy Tonatiuh	48317	FEAT(UI): Agregar boton para regresar al menu con alerta de confirmacion
06/10/2026 20:34	Soy Tonatiuh	b253c	FIX(UI): Iniciar la partida de ejemplo con el turno del Jugador 1

Fecha	Autor	Commit	Mensaje
06/10/2026 21:00	Soy Tonatiuh	b7b07	FEAT(UI): Agregar selector de color para comodines
07/10/2026 12:50	Soy Tonatiuh	5b983	FEAT(UI): Agregar boton para cantar UNO frente a cada jugador
07/10/2026 16:09	Soy Tonatiuh	66b36	FEAT(UI): Agregar pantalla de victoria
07/10/2026 16:38	Soy Tonatiuh	26fcb	FIX(UI): Quitar la fuente Inter que no se usaba y fijar el tema oscuro
07/10/2026 16:44	Soy Tonatiuh	6750d	FIX(UI): Usar CardColor para el color activo
07/10/2026 17:16	Soy Tonatiuh	0d75c	FIX(UI): Ordenar BoardViewModel
07/10/2026 18:44	Rodrigo Benitez	213b8	feat(GameEngine): se implementan los atributos de la clase GameEngine
07/10/2026 18:47	Rodrigo Benitez	2f999	feat(GameEngine): Implmenta getters de GameEngine
07/10/2026 19:11	Rodrigo Benitez	fbe68	feat(GameEngine): Implementa el metodo StartGame que limpia y comienza la partida
07/10/2026 19:16	Rodrigo Benitez	2417f	feat(GameEngine): agrega todas las funciones al metodo StartGame
07/10/2026 19:25	Rodrigo Benitez	67e3e	feat(GameEngine): se agregan los metodos para saber si una carta puede ser jugada y el metodo para ejecutar la carta
07/10/2026 19:50	Rodrigo Benitez	3e7b8	feat(GameEngine): metodos para robar, pasar turno y cantar UNO
07/10/2026 20:10	Rodrigo Benitez	9f81e	feat(GameEngine): agrega metodos privados que ya eran usados por los metodos publicos
07/10/2026 20:14	Rodrigo Benitez	cdd98	feat(GameEngine): agrega metodos para repartir cartas a los jugadores y revisar si el jugador tiene penalizaciones(si no cantó UNO)
07/10/2026 20:30	Rodrigo Benitez	5564c	feat(GameEngine): agrega funciones para monitoreo de jugadas y para terminar la partida
07/10/2026 20:34	Rodrigo Benitez	db064	fix(Player.cs): Agrega metodos faltantes en la clase Player.cs
07/10/2026 23:49	Alejandro	148df	feat(infrastructure): Añadi la entidad de MatchPlayers para guardar los jugadores de una cierta partida, esto trajo cambios a la match. Agregue ApiGameLog y ApiMatch repositorios, estos para recibir los datos de la api, recibe el json y los convierte para su uso. Se hizo un cambio en MatchRepository para guardar los id de los jugadores. En GameAuditService se modifico para que cada partida nueva se reinicie la suscripcion a los eventos. Se agrego la configuracion de la matchplayers a UnoDbContext. Se agrego la conexion de la api a la app principal en App.axaml.cs
08/10/2026 19:46	Rodrigo Benitez	52d20	feat(Events): Agrega la funcion notificadora de la carta jugada
08/10/2026 19:49	Rodrigo Benitez	8b97f	fix(Events): Elimina un archivo obsoleto que funcionó para declarar la arquitectura
08/10/2026 19:52	Rodrigo Benitez	144e7	feat(Events): implementa las clases previamente declaradas en GameEvents
08/10/2026 20:05	Rodrigo Benitez	7972a	fix(Events): Por coherencia con la arquitectura separa IGameLogRepository.cs en 3 clases. una interfaz, un enum y un DTO
08/10/2026 20:08	Rodrigo Benitez	8d638	feat(GameEngine): Al terminar una partida hace un logaction hacia la BD
08/10/2026 20:12	Héctor Rodrigo Benítez Gama	2f4f6	Merge pull request #4 from rodrigo-bg/Interfaz
08/10/2026 20:15	Héctor Rodrigo Benítez Gama	2cd15	Merge branch 'main' into Infrastructure
08/10/2026 20:18	Héctor Rodrigo Benítez Gama	02cbd	Merge pull request #5 from rodrigo-bg/Infrastructure

Fecha	Autor	Commit	Mensaje
08/10/2026 20:26	Rodrigo Benitez	7cef5	Merge branch 'main' into Core
08/10/2026 20:40	Rodrigo Benitez	bf0f9	fix(Core): arregla algunas discrepancias con Infrastructure en el nombramiento de las variables de GameLogEntryDto.cs y metodos faltantes
08/10/2026 20:45	Héctor Rodrigo Benítez Gama	8d590	Merge pull request #6 from rodrigo-bg/Core
09/10/2026 10:22	Soy Tonatiuh	2ecb7	FEAT(UI): Iniciar la partida con el motor de juego
09/10/2026 10:40	Soy Tonatiuh	98303	FEAT(UI): Tirar cartas con un clic
09/10/2026 11:01	Soy Tonatiuh	49b2d	FEAT(UI): Robar del mazo y pasar turno
09/10/2026 11:45	Soy Tonatiuh	11fcd	FEAT(UI): Oscurecer las cartas que no se pueden tirar
09/10/2026 11:48	Soy Tonatiuh	98a4e	FIX(UI): Rediseño de cartas 8 bit
09/10/2026 11:54	Alejandro	a9f79	fix(infrastructure): Cambios minimos para MatchPlayer
09/10/2026 11:56	Alejandro	a9bb0	Merge branch 'main' into Infrastructure Union de UI y Core
09/10/2026 12:03	Soy Tonatiuh	bbd83	FIX(UI): Avisar al motor cuando un jugador canta UNO
09/10/2026 12:09	Soy Tonatiuh	e8a97	FEAT(UI): Mostrar la pantalla de victoria al terminar la partida
09/10/2026 12:18	Soy Tonatiuh	b8532	FEAT(UI): Dar 2 segundos para cantar UNO despues de tirar la penultima carta
09/10/2026 12:30	Soy Tonatiuh	52cbc	FEAT(UI): Avisar la penalizacion por no cantar UNO
09/10/2026 12:36	Soy Tonatiuh	8eb5a	FEAT(UI): Agregar pantalla de historial con datos de ejemplo
09/10/2026 12:42	Soy Tonatiuh	4e5bc	FEAT(UI): Abrir el historial desde el menu
09/10/2026 12:54	Soy Tonatiuh	7ddfa	FIX(UI): Recibir el motor de juego en el constructor de BoardViewModel
09/10/2026 12:55	Soy Tonatiuh	e5fb4	FEAT(UI): Mostrar duracion y puntos en la pantalla de victoria
09/10/2026 15:21	Héctor Rodrigo Benítez Gama	75ff0	Merge pull request #10 from rodrigo-bg/Interfaz
09/10/2026 15:37	Rodrigo Benitez	15f4b	feat(test): crea las pruebas unitarias para validr el funcionamiento del mazo
09/10/2026 15:58	Rodrigo Benitez	781e9	feat(test): Agrega la clase de pruebas unitarias para GameRules
10/10/2026 00:46	Alejandro	3107c	fix(infrastructure): Se aseguro que la claves primarias existan en MatchId y PlayerId. Se configuro para que se cargue la semilla correctamente. Se uso solo GameEngine para el registro de auditoria y evitar duplicados. Se amplio PlayerStatsDto para guardar duración promedio y puntajes de partida. Se realiza el registro completo de las partidas(cartas jugadas, turnos, penalizaciones. ganador, duración)
10/10/2026 10:03	Tony_Code	764af	Merge pull request #11 from rodrigo-bg/Infrastructure
10/10/2026 10:24	Soy Tonatiuh	6e524	FIX(UI): Ajustar el historial al nuevo formato de estadisticas
10/10/2026 10:42	Soy Tonatiuh	8b632	FEAT(UI): Conectar el motor y la API para guardar las partidas
10/10/2026 11:58	Soy Tonatiuh	87751	FEAT(UI): Guardar las partidas en la API y mostrar el historial real
10/10/2026 12:51	Soy Tonatiuh	96a89	FEAT(UI): Mostrar la lista de partidas y la bitacora de movimientos en el historial
10/10/2026 14:42	Soy Tonatiuh	c0066	FEAT(UI): Agregar README con los pasos para ejecutar el juego y la API
10/10/2026 21:29	Héctor Rodrigo Benítez Gama	2ba63	Merge pull request #12 from rodrigo-bg/Interfaz

Fecha	Autor	Commit	Mensaje
11/10/2026 12:49	Rodrigo Benitez	a9f93	feat(tests): agrega clase para pruenas unitarias de GameEngine
11/10/2026 21:12	Alesigorotiro	50909	Add files via upload
01/11/2026 10:52	Soy Tonatiuh	635cf	FEAT(UI): Mostrar las opciones crear y unirse a partida en el inicio
01/11/2026 11:17	Soy Tonatiuh	3be71	FEAT(UI): Agregar el servicio de sala y la IP local
01/11/2026 11:57	Soy Tonatiuh	da23f	FEAT(UI): Agregar las pantallas para crear y unirse a partida
01/11/2026 12:32	Soy Tonatiuh	37bcf	FEAT(UI): Agregar la sala de conexión con los jugadores unidos
01/11/2026 12:57	Soy Tonatiuh	73d9f	FEAT(UI): Conectar el inicio, la sala y la partida
01/11/2026 13:19	Soy Tonatiuh	fc3f3	FEAT(UI): Agregar sonido al picar los botones
01/11/2026 13:41	Soy Tonatiuh	45d6e	FEAT(UI): Agregar sonido al tirar y robar cartas
01/11/2026 13:51	Soy Tonatiuh	769f4	FIX(UI): Bajar el volumen de la musica
01/11/2026 14:16	Soy Tonatiuh	3831c	FEAT(UI): Agregar sonidos de victoria y derrota
01/11/2026 14:17	Soy Tonatiuh	c7f3f	FIX(UI): Regresar a la sala al volver a jugar
01/11/2026 14:29	Soy Tonatiuh	9efcc	FIX(UI): Mostrar la pantalla final en 1.5 segundos y pausar la música
01/11/2026 14:29	Soy Tonatiuh	3bb31	FEAT(UI): Pausar y reanudar la música
03/11/2026 10:32	Héctor Rodrigo Benítez Gama	bf80a	Merge pull request #13 from rodrigo-bg/Interfaz
03/11/2026 15:49	Rodrigo Benitez	62bca	chore: actualiza .gitignore con carpetas locales
03/11/2026 15:55	Rodrigo Benitez	bb194	chore(test): elimina archivo plantilla UnitTest1
03/11/2026 18:33	Rodrigo Benitez	36717	feat(server): inicializa proyecto TheUno.Server con SignalR y Kestrel en puerto 5000
04/11/2026 08:57	Soy Tonatiuh	fa7ba	FEAT(UI): Agregar SignalR y la conexión al servidor
04/11/2026 09:22	Soy Tonatiuh	8b1f6	FEAT(UI): Agregar los mensajes del contrato con el servidor
04/11/2026 09:57	Soy Tonatiuh	18e46	FEAT(UI): Agregar el servicio de sala con SignalR
04/11/2026 10:17	Soy Tonatiuh	b948c	FEAT(UI): Conectar la sala al servidor
04/11/2026 10:47	Soy Tonatiuh	36c6d	FEAT(UI): Agregar la sesión de juego
04/11/2026 11:07	Soy Tonatiuh	8487e	FEAT(UI): Entregar la sesión de juego desde la sala
04/11/2026 11:47	Soy Tonatiuh	7f5ea	FIX(UI): Alimentar el tablero con el estado del servidor
04/11/2026 12:17	Soy Tonatiuh	bc66e	FEAT(UI): Mostrar las manos rivales con el reverso
04/11/2026 12:47	Soy Tonatiuh	034f2	FIX(UI): Enviar las jugadas al servidor
04/11/2026 13:22	Soy Tonatiuh	9517d	FEAT(UI): Avisar si se pierde la conexión
04/11/2026 13:42	Soy Tonatiuh	f7339	FIX(UI): Ajustar el ancho de los botones de jugador
04/11/2026 14:07	Soy Tonatiuh	eb507	FEAT(UI): Agregar el silencio de la música y los efectos
04/11/2026 14:37	Soy Tonatiuh	22fe4	FEAT(UI): Agregar el botón de sonido en el menú
04/11/2026 14:57	Soy Tonatiuh	0dfc2	FEAT(UI): Agregar la pantalla completa con F11
04/11/2026 17:41	Rodrigo Benitez	41a62	Merge remote-tracking branch 'origin/Interfaz' into Server # Please enter a commit message to explain why this merge is necessary, # especially if it merges an updated upstream into a topic branch. # # Lines starting with '#' will be ignored, and an empty message aborts # the commit.
04/11/2026 17:59	Rodrigo Benitez	68599	feat(server): agrega DTOs de red CardDto, PlayerStateDto y GameStateDto

Fecha	Autor	Commit	Mensaje
04/11/2026 18:03	Rodrigo Benitez	9263c	feat(server): define constantes de protocolo HubMethods y HubMessages
04/11/2026 18:08	Rodrigo Benitez	ffe1f	feat(server): implementa GameSessionManager para gestionar sala y motor en memoria
04/11/2026 18:11	Rodrigo Benitez	2f491	feat(hub): implementa GameHub con metodos de sala, partida y difusion en tiempo real
04/11/2026 18:15	Rodrigo Benitez	f8920	feat(server): registra GameSessionManager y mapea GameHub en ruta /gamehub
04/11/2026 18:18	Rodrigo Benitez	aab19	fix(ui): permite detectar IP local en conexiones Ethernet directas sin puerta de enlace
04/11/2026 18:24	Rodrigo Benitez	944a2	test(server): agrega pruebas unitarias para GameSessionManager y filtrado de manos
05/11/2026 11:43	Rodrigo Benitez	f52cf	fix(server): cambia el puerto a 5050 porque en macOS el 5000 lo ocupa AirPlay
05/11/2026 11:44	Rodrigo Benitez	d609c	test(hub): agrega prueba de integracion con 3 clientes SignalR contra el GameHub
06/11/2026 09:08	Alejandro	dd932	feat(infrastrcuture): Migracion del servicio de auditoria, se actualizaron las referencias
06/11/2026 10:37	Alejandro	dea18	fix(Infrastructure): modelado de datos
06/11/2026 10:48	Alejandro	ba77d	fix(infratructure): Cambios en GameAuditService para la conexion al servidor
06/11/2026 10:52	Alejandro	a070e	fix(infrastructure): Cambio en la ip del readme
06/11/2026 10:56	Alejandro	73921	fix(infrastructure): REconfiguración de clientes HTTP y mantenimiento de contratos
06/11/2026 11:05	Alejandro	b5ea3	Compatibilidad de JSON
06/11/2026 11:51	Tony_Code	813d2	Merge pull request #15 from rodrigo-bg/Infrastructure
06/11/2026 11:51	Tony_Code	78631	Merge pull request #14 from rodrigo-bg/Interfaz
07/11/2026 16:12	Soy Tonatiuh	51599	FIX(UI): Regresar de la sala a las opciones de crear y unirse
07/11/2026 16:38	Soy Tonatiuh	6eb5a	FEAT(UI): Recordar la IP del servidor de la partida
07/11/2026 17:14	Soy Tonatiuh	cd8b6	FEAT(UI): Leer el historial de la API del servidor
07/11/2026 17:38	Soy Tonatiuh	6445c	FEAT(UI): Mostrar y cambiar la IP del historial
08/11/2026 09:31	Soy Tonatiuh	3df1f	FEAT(UI): Avisar si ya existe una partida creada
08/11/2026 09:58	Soy Tonatiuh	203b4	FIX(UI): Usar los nombres de jugador del historial
08/11/2026 10:26	Soy Tonatiuh	405f3	FIX(UI): Ajustar los avisos largos de la sala
08/11/2026 10:47	Soy Tonatiuh	c38a4	FIX(UI): Guardar el número de cada carta en la bitácora
08/11/2026 11:09	Soy Tonatiuh	1ae6f	FIX(UI): Explicar en el README cómo jugar en distintas computadoras
08/11/2026 11:22	Tony_Code	a33fd	Merge pull request #16 from rodrigo-bg/Interfaz
08/11/2026 14:31	Soy Tonatiuh	aba99	FIX(UI): Quitar del cliente el registro de la API
08/11/2026 14:56	Soy Tonatiuh	a3d0b	FIX(UI): Ordenar y comentar los servicios de la sala
08/11/2026 15:30	Soy Tonatiuh	a1cb8	FIX(UI): Comentar los contratos de red
08/11/2026 15:52	Soy Tonatiuh	96a66	FIX(UI): Ordenar el audio y quitar código repetido
08/11/2026 16:30	Soy Tonatiuh	8a9c2	FIX(UI): Ordenar el tablero y quitar código repetido

Fecha	Autor	Commit	Mensaje
08/11/2026 16:33	Rodrigo Benitez	22a66	Merge branch 'main' into Server
08/11/2026 16:40	Rodrigo Benitez	2a30f	feat(server): sincroniza nombres de jugadores, rechazo de sala duplicada y DI con PR #15 y #16
08/11/2026 16:55	Soy Tonatiuh	0286b	FIX(UI): Ordenar y comentar la sala y la navegación
08/11/2026 17:21	Soy Tonatiuh	b65f6	FIX(UI): Comentar los jugadores y ordenar los avisos del historial
08/11/2026 17:37	Tony_Code	ce6f9	Merge pull request #17 from rodrigo-bg/Interfaz
08/11/2026 18:31	Héctor Rodrigo Benítez Gama	66445	Merge pull request #18 from rodrigo-bg/Server
08/11/2026 19:26	Soy Tonatiuh	2f642	FEAT(UI): Animar a los rivales mientras eligen carta
08/11/2026 19:30	Soy Tonatiuh	09999	FEAT(UI): Avisar a todos cuando se tira una carta especial
08/11/2026 19:32	Soy Tonatiuh	52663	FEAT(UI): Resaltar en amarillo el cambio de dirección
08/11/2026 19:44	Soy Tonatiuh	5cb80	FEAT(UI): Agrandar el nombre del jugador en turno
08/11/2026 19:47	Rodrigo Benitez	cea85	fix(ui): evita bloqueo del hilo principal en macOS al volver a jugar y pausar afplay
08/11/2026 19:53	Soy Tonatiuh	f9818	FIX(UI): Hacer más notoria la animación de los rivales
08/11/2026 19:54	Soy Tonatiuh	69f2e	FIX(UI): Agrandar a 1.5 veces el nombre del jugador en turno
08/11/2026 19:58	Tony_Code	56656	Merge pull request #20 from rodrigo-bg/Interfaz
08/11/2026 20:08	Héctor Rodrigo Benítez Gama	aab57	Merge pull request #19 from rodrigo-bg/Server
08/11/2026 20:08	Héctor Rodrigo Benítez Gama	7308b	Merge branch 'main' into Server
08/11/2026 20:13	Tony_Code	c7998	Merge pull request #21 from rodrigo-bg/Interfaz
08/11/2026 20:13	Soy Tonatiuh	f3129	FIX(UI): Detener la animación de los rivales al salir de la partida
08/11/2026 21:26	Soy Tonatiuh	71c9f	FIX(UI): Regresar la animación de los rivales y ajustar nombres y avisos del tablero
08/11/2026 21:31	Soy Tonatiuh	71427	FIX(UI): Cambiar de jugador en la sala, volver a jugar con el mismo y quitar los puntos de la bitácora
08/11/2026 21:40	Soy Tonatiuh	8bd78	FEAT(UI): Avisar a todos cuando alguien canta UNO o no lo canta
08/11/2026 21:40	Soy Tonatiuh	969ce	FIX(UI): Mostrar en mayúsculas solo el nombre del jugador en turno
08/11/2026 21:47	Tony_Code	a6bc3	Merge pull request #22 from rodrigo-bg/Interfaz

5. Listado de ramas generadas

Trabajamos con una rama por área y la rama main como rama principal. En la fase 2 se agregó la rama Server para el servidor central. Cada rama se integró a main por medio de pull requests; antes de cada push y de cada pull request, cada integrante traía los cambios de main a su rama para resolver los conflictos de su lado.

Rama	Responsable	Uso
main	Todo el equipo	Rama principal; recibe los cambios de las demás ramas por pull request.
Core	Rodri	Motor de juego (TheUno.Core). Se eliminó del repositorio después de integrarse a main.
Infrastructure	Alejandro	Base de datos, repositorios, auditoría y API en red (TheUno.Infrastructure).
Interfaz	Tona	Interfaz gráfica y cliente en red (TheUno.UI).
Server	Rodri	Servidor central con SignalR (TheUno.Server) y pruebas (TheUno.Tests).

La figura siguiente resume el flujo que seguimos con cada cambio, desde la rama propia hasta main y de regreso:

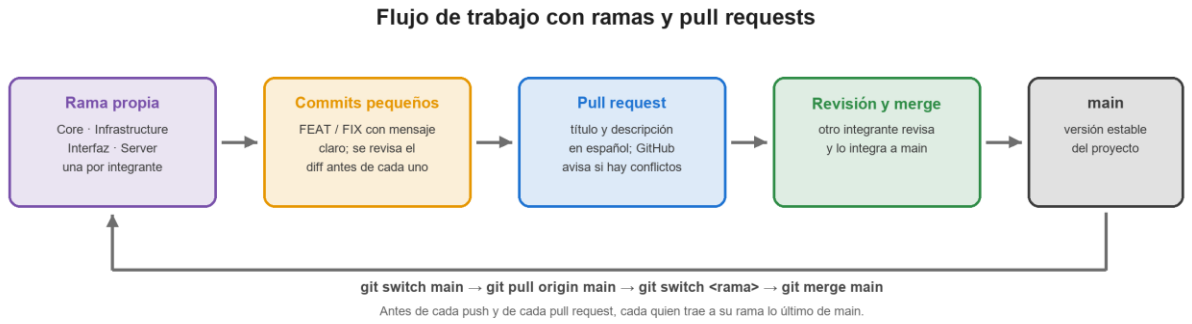


Figura 11. *Flujo de trabajo con ramas y pull requests.*

Pull requests integrados a main, en orden:

Fecha	Rama	PR	Contenido
01/10/2026	Core	#1	Modelos de carta y jugador, eventos del juego y contratos para la interfaz.
05/10/2026	Interfaz	#2	Tablero de 8 bits, menú principal, música y cartas conectadas con el Core.
06/10/2026	Core	#3	Baraja (barajar, robar y descartar) y reglas oficiales (jugada legal, +4 y sin acumulación).
08/10/2026	Interfaz	#4	Selector de color, botón ¡UNO! y pantalla de victoria.

Fecha	Rama	PR	Contenido
08/10/2026	Infrastructure	#5	API en Python, GameAuditService, entidad MatchPlayer y corrección de migraciones.
08/10/2026	Core	#6	Eventos del motor, registro de acciones y ajustes de contratos con Infrastructure.
09/10/2026	Interfaz	#10	Ventana de 2 segundos para cantar UNO, penalización e historial.
10/10/2026	Infrastructure	#11	Llaves primarias de MatchPlayer y semilla de jugadores.
10/10/2026	Interfaz	#12	Partidas guardadas en la API, historial real y README.
03/11/2026	Interfaz	#13	Pantallas para crear y unirse a partida, sala de conexión y efectos de sonido.
06/11/2026	Interfaz	#14	Jugadas enviadas al servidor, aviso de conexión perdida, botón de sonido y F11.
06/11/2026	Infrastructure	#15	Auditoría en el servidor, clientes HTTP reconfigurados y compatibilidad de JSON.
08/11/2026	Interfaz	#16	Historial con la IP del servidor, aviso de partida existente y README para jugar en red.
08/11/2026	Interfaz	#17	Limpieza del cliente y registro de la API solo en el servidor.
08/11/2026	Server	#18	Servidor SignalR (GameHub y GameSessionManager), puerto 5050 y pruebas.
08/11/2026	Interfaz	#20	Avisos de cartas especiales, animación de los rivales y cambio de dirección resaltado.
08/11/2026	Server	#19	Salida en segundo plano para evitar el bloqueo en macOS al volver a jugar.
08/11/2026	Interfaz	#21	Detener la animación de los rivales al salir de la partida.
08/11/2026	Interfaz	#22	Avisos de UNO, nombre del jugador en turno, cambiar de jugador y volver a jugar.

6. Problemas encontrados y cómo los solucionamos

6.1 Sobre el proyecto

Motor de juego (Rodri)

- **Contratos distintos:** GameAuditService fallaba por los nombres de los parámetros de GameLogEntryDto; se ajustó el constructor del DTO para aceptar las dos formas.
- **Nombres de eventos:** Infrastructure esperaba otros nombres de propiedades; se agregaron alias en los argumentos de los eventos.
- **Pruebas desactualizadas:** se adaptaron a la firma real de GameRules para validar bien la restricción del +4.

Interfaz (Tona)

- **Música en Windows y macOS:** la primera versión usaba LibVLC y no funcionaba en macOS. La cambiamos por NAudio en Windows y el comando afplay en macOS.
- **Cartas borrosas:** Avalonia suavizaba las imágenes en pixel art. Desactivamos la interpolación con RenderOptions.BitmapInterpolationMode, que tiene que ir en el elemento raíz de la vista.
- **Cantar UNO:** el botón aparece cuando el jugador ya tiró su penúltima carta y el turno es del siguiente, así que el método del motor para cantar UNO no le aplicaba. La interfaz da 2 segundos, marca al jugador que cantó y guarda el movimiento en el log.

Base de datos y API (Alejandro)

- **Llave de MatchPlayer:** la tabla de jugadores por partida no tenía llave primaria y la base de datos no se podía crear. Se definió una llave compuesta (MatchId, PlayerId).
- **La API y C# no se entendían:** la API responde en snake_case (match_id) y C# esperaba MatchId, así que el Id de la partida llegaba vacío. Se configuró la lectura con JsonNamingPolicy.SnakeCaseLower.
- **Movimientos rechazados por la base:** el motor creaba jugadores con identificadores nuevos en cada partida y la base los rechazaba por las llaves foráneas. Ahora la partida inicia con los jugadores registrados y con el Id de la partida creada en la API.

6.2 Sobre la migración a red local

Servidor central (Rodri)

- **Puerto ocupado en macOS:** el puerto 5000 lo usa AirPlay en macOS y el servidor no arrancaba. Se movió al puerto 5050 y se actualizó el cliente.

- **Segunda partida en el mismo servidor:** si otra computadora intentaba crear una partida, el servidor la rechaza con una HubException y el cliente muestra "Partida existente ya creada con la misma red IP".
- **Bloqueo en macOS al volver a jugar:** salir de la partida esperaba al servidor en el hilo principal y la música se trababa. La salida ahora se hace en segundo plano.

Cliente de interfaz (Tona)

- **Tablero vacío al iniciar:** el aviso de inicio podía llegar antes que la primera mesa. El cliente espera la primera mesa antes de abrir el tablero.
- **Cartas de los rivales:** el tablero ya no recibe las manos de los demás; las dibuja boca abajo según cuántas cartas tienen.
- **Animación que seguía corriendo:** al salir a media partida, el temporizador de la animación de los rivales seguía activo. Ahora se detiene al salir.

Persistencia en red (Alejandro)

- **API solo en la misma computadora:** uvicorn escucha por defecto en 127.0.0.1 y las otras computadoras no podían consultarla. Se arranca con --host 0.0.0.0.
- **Partidas registradas dos veces:** el cliente y el servidor registraban la misma partida en la API. Ahora solo la registra el servidor, junto con la auditoría.
- **Firewall y redes públicas:** con los datos de un celular, Windows marca la red como pública y bloquea las conexiones. Hay que permitir los puertos 5050 y 8000 o marcar la red como privada.

6.3 Sobre el uso de Git y GitHub

- **Saltos de línea entre sistemas:** trabajamos desde Windows y macOS, y los saltos de línea distintos generaban cambios falsos. Los normalizamos con un archivo .gitattributes.
- **Conflictos al integrar:** al unir las ramas surgieron conflictos, sobre todo en las migraciones de la base de datos y en los nombres de los contratos. Los resolvimos en la rama de cada integrante antes de abrir el pull request.
- **Pull request que rompió la compilación:** un cambio en un contrato del Core llegó a main y la interfaz dejó de compilar. Lo corregimos con un commit de ajuste en la rama Interfaz.
- **Cambios que se colaban en los commits:** al editar archivos mientras se preparaba un commit, git add . incluía cambios que no correspondían. Empezamos a revisar el diff completo antes de cada commit.
- **Commits demasiado grandes:** un refactor de muchos archivos en un solo commit era difícil de revisar y se revirtió. Trabajamos con commits pequeños y mensajes con prefijos como feat y fix.

7. Conclusión

Desarrollamos un juego de UNO completo que sigue las reglas oficiales, se juega entre tres computadoras de la misma red y guarda jugadores, historial y bitácora en una base de datos. Separar el proyecto en motor, servidor, interfaz y persistencia, con contratos definidos desde el inicio, nos permitió trabajar en paralelo en las dos fases.

Lo más difícil fue la integración: en la fase 1, los nombres de los contratos y los identificadores entre el motor y la base; en la fase 2, acordar el contrato del hub, decidir qué información viaja a cada computadora y resolver detalles de red como los puertos de macOS y el firewall de Windows. Con una rama por área y 19 pull requests revisados aprendimos a mantener las ramas al día y a hacer commits pequeños y claros.